

Guía de ejercicios

IA Sup-rate

Teoría, ejercicios, entregables y rúbricas para cada una de las 10 semanas. Las semanas 1 y 2 son de teoría y desmitificación; de la 3 a la 8 construyes modelos de Machine Learning; las 9 y 10 te convierten en desarrollador/a web.

 16–18 años

 Equipos de 4

 10 semanas

 2 semanas de teoría

 6 semanas de Machine Learning

 2 semanas de web

Desmitificación

Semanas 1–2 · Qué es y qué no es la IA, mitos, IA generativa e ingeniería de prompts.

Machine Learning

Semanas 3–8 · Regresión, árboles, KNN, K-Means, redes neuronales y NLP con Python.

Desarrollo Web

Semanas 9–10 · Portafolio con Next.js + TypeScript, APIs y despliegue en Vercel.



Semana	Tema	Fase	Qué harás
1	Qué es la IA de verdad · Cazadores de mitos	Teoría	Clasificar 10 mitos/realidades, vocabulario base, Teachable Machine, cuestionario con PDF de reporte
2	IA Generativa · Prompt Battle	Teoría	Quiz de tokens/embeddings/sesgo y redacción de un prompt perfecto (rol+contexto+restricciones)
3	Regresión Lineal · predice tu nota	ML	Entrenar un modelo que predice un número y medir MAE / R^2
4	Árbol de Decisión · Titanic	ML	Modelo que decide si alguien sobrevive e interpretar su raíz
5	KNN · recomendador de películas	ML	Recomendar películas con los K vecinos más cercanos y escalar datos
6	K-Means · segmentación de clientes	ML	Descubrir grupos sin etiquetas y nombrar cada segmento
7	Redes Neuronales · dígitos MNIST	ML	Entrenar una red que lee números a mano y experimentar arquitecturas
8	NLP · análisis de sentimiento	ML	Distinguir reseñas positivas/negativas con vectorización de texto
9	Portafolio web con Next.js	Web	Personalizar componentes, datos y estilos hasta dejar «Build OK»
10	API + Vercel	Web	Conectar GET/POST y publicar tu portafolio con un link público

Cómo usar esta guía (por semana)

1. **Lee la teoría** de la semana (semanas 1–2) o la explicación corta del modelo (semanas 3–8).
2. **Resuelve los ejercicios** en equipo: las hojas de trabajo y las marcas **TODO** del código.
3. **Completa los entregables** señalados y sube todo a GitHub en la rama `equipo-N-semana-XX`.
4. **Marca el autochequeo** al final de cada semana antes de presentar.
5. **Presenta en 3 minutos** y rota el rol de capitán/integrador, investigador, código y presentador cada semana.

Regla de oro

Nadie te da la respuesta final: la guía te da la **estructura, las pistas y el camino**. La magia la haces tú con tu equipo.

👥 Equipos de 4 y roles rotativos

Rol	Responsabilidad	Rótalo
👑 Capitán / Integrador	Organiza, junta las piezas del equipo y sube el reto a GitHub	Cada semana
🔍 Investigador(a)	Busca teoría, documenta y cita fuentes (revisa <code>tools/</code>)	Cada semana
🔧 Código / Datos	Se encarga del código, la limpieza de datos y las pruebas	Cada semana
📢 Presentador(a)	Prepara la explicación final y defiende el reto ante el grupo	Cada semana

Cada semana debes rotar de rol mínimo 1 vez en las 10 semanas. Anota en cada entrega quién hizo qué.

✅ Checklist general de entrega

- ☐ Cloné el repositorio con `tools/01-git-y-github.md`
- ☐ Leí el `README.md` de mi semana
- ☐ Completé las marcas `TODO` del código en equipo
- ☐ El código corre y el autochequeo (si lo tiene) pasa
- ☐ Escribí en `ENTREGA.md` quiénes participaron y qué hizo cada quien
- ☐ Hice `commit` y `push` a la rama `equipo-N-semana-XX`
- ☐ Presenté el reto en 3 minutos al grupo

📁 Glosario rápido (lea antes de la semana 1)

Concepto	En cristiano
IA	Programas que aprenden de ejemplos para predecir o decidir. NO piensa, NO siente, NO tiene consciencia.
IA simbólica	La vieja escuela: reglas escritas a mano por humanos («SI llueve ENTONCES lleva paraguas»).
IA conexionista	La escuela moderna: la máquina aprende patrones desde millones de ejemplos (redes neuronales).
Modelo	El «cerebro» ya entrenado; la máquina de predicciones.
Entrenamiento	El proceso donde el modelo ve ejemplos y ajusta sus conexiones.
Machine Learning (ML)	Programas que mejoran su precisión con datos, sin ser programados regla por regla.
Regresión / Clasificación	Predecir un NÚMERO / predecir una CATEGORÍA.
Feature (característica)	Cada columna que describe el dato (edad, horas de estudio...).
Datos de entrenamiento / prueba	Partes del set: una para aprender y otra para COMPROBAR que no memorizó.
Overfitting (sobreajuste)	El modelo «memoriza» los ejemplos y falla con datos nuevos.
Token / Embedding / Atención	Unidad mínima de texto / palabras en números / qué palabras importan más.
Prompt	La instrucción que le escribes a la IA. Buen prompt = buena respuesta.

Alucinación	La IA inventa datos falsos pero confiados: NO «miente», completa texto probable. ¡Verifica siempre!
Sesgo (bias)	El modelo hereda prejuicios de los datos mal balanceados con que se entrenó.
Componente / API / JSON / Deploy	Pieza reutilizable / ventanilla que devuelve datos / formato de datos / subir tu app a internet.



¿Qué es la IA de verdad? (Rompe el mito)

Misión: convertirte en un/a **Cazador/a de Mitos** de la IA. Tu equipo debe separar lo que la IA **sí** es de lo que **solo parece** en las películas. Al final se sube a GitHub y se presenta en 3 minutos.



Un solo intento

El cuestionario `index.html` pide **nombre, equipo e integrantes** antes de empezar. Solo se puede realizar **UNA vez**; al terminar se bloquea y debes **descargar el PDF** con tus aciertos y errores. Ese PDF es **tu evidencia**.

A Teoría de la semana (estúdiala antes de los ejercicios)



¿Qué es (y qué NO es) la Inteligencia Artificial?

- La IA es un **programa que aprende de ejemplos** para predecir o decidir.
- **NO piensa** como tú, **NO siente** y **NO tiene consciencia**. No hay «misterio»: hay patrones y matemática.
- **IA simbólica** (la vieja escuela): reglas escritas a mano por humanos, ej. «Si llueve ENTONCES lleva paraguas».
- **IA conexionista** (la moderna): aprende patrones desde millones de ejemplos. Ej.: las redes neuronales.
- **Modelo** es el «cerebro» ya entrenado; **entrenamiento** es el proceso en que ve ejemplos y ajusta sus conexiones.
- Un **clasificador** solo aprende a partir de **muchos ejemplos etiquetados** (foto + «esto es un perro»).



¿De dónde sale el sesgo?

El sesgo NO suele nacer de un programador malvado: nace de **datos mal balanceados**. Si entrenamos un modelo solo con datos de un grupo, aprenderá a favorecer a ese grupo. Por eso revisamos los datos, no solo la «magia».

Lee también `tools/05-vocabulario-ia.md` (filas de la Semana 1) y contesta en equipo:

- ¿Cuál es la diferencia entre **IA simbólica** e **IA conexionista**?
- ¿La IA «piensa»? ¿Siente? ¿Es consciente?
- ¿Cómo se «entrena» un modelo? Busquen la palabra **entrenamiento** en el glosario.

B Ejercicio 1 · Cazadores de mitos 🔍

Clasifica cada afirmación como **MITO** o **REALIDAD** y justifica con al menos 1 línea

#	Afirmación	MITO / REALIDAD	Justificación (1 línea)
1	«La IA puede sentir emociones como un humano.»		
2	«Los asistentes como Siri, Alexa o Google usan IA para entenderte.»		
3	«La IA es una sola tecnología: si la usaste una vez, ya conociste toda la IA que existe.»		
4	«Una IA puede inventar respuestas falsas pero decirlas con total seguridad.» (pista: alucinación)		
5	«Para crear un modelo de IA hace falta saber programar en Python.»		
6	«Los filtros de spam de tu correo ya usan Machine Learning.»		
7	«La IA de hoy funciona con reglas escritas a mano por programadores.» (¿simbólica o conexionista?)		
8	«Un modelo entrenado como clasificador solo aprende a partir de muchos ejemplos etiquetados.»		
9	«Ninguna IA puede hoy escribir, dibujar o componer música.»		
10	«El sesgo nace de un humano que programó a propósito discriminar a un grupo.» (ve el glosario 🗨️)		

💡 Las afirmaciones 1–6 tratan mitos generales; 7–10 te empujan a usar el vocabulario de la Semana 1.

C Ejercicio 2 · Entrena un mini-modelo 🤖

Teachable Machine de Google

1. Abre <https://teachablemachine.withgoogle.com>.
2. Entrena un modelo mini con tu cámara: **3 clases de ~10 fotos** cada una.
3. Comprueba que no es «mágico»: es un **clasificador que aprende patrones de las fotos**, igual que el cuestionario aprende de ti.
4. Guarda captura o link como **evidencia**.

⚠️ Si internet no coopera, captura la pantalla y preséntala después. El reto principal no depende de esta actividad.

Evidencia / captura del modelo: _____

D Cuestionario · Reporte en PDF

Abre [semana-01/index.html](#) en el navegador, regístrate con tu nombre y los integrantes del equipo, responde el cuestionario y **descarga el PDF** (botón «Descargar reporte»). Ese PDF se sube a la carpeta junto con tu entrega.

E Entregables (qué subir a GitHub)

1. `semana-01/RESPUESTAS.md` → nombres del equipo + rol de cada quien.
2. Tabla con tus decisiones: MITO / REALIDAD + justificación.
3. Evidencia de Teachable Machine (link o imagen).
4. `reporte-semana1.pdf` descargado del cuestionario.

F Rúbrica (20 puntos)

Criterio	Puntos
Decisiones correctas (MITO / REALIDAD)	8
Justificaciones claras (usan vocabulario de la S1)	6
Evidencia Teachable Machine	3
Presentación de 3 min (clara y en equipo)	3

G Autochequeo del equipo

- ☐ Respondí las 10 afirmaciones con justificación.
- ☐ Defino con mis palabras: simbólica vs. conexionista, entrenamiento, sesgo.
- ☐ Mi equipo entrena un modelo en Teachable Machine (o lo captura).
- ☐ Todos hicimos el cuestionario de `index.html` registrando nuestros datos.
- ☐ Descargamos el **PDF del reporte** (aciertos y errores).
- ☐ `RESPUESTAS.md` subido en la rama `equipo-N-semana-01`.



Ritual de cierre

3 minutos para explicar el mito que más les costó derribar y por qué.



IA Generativa: tu copiloto (Prompt Battle)

Misión: entender cómo «piensa» una IA generativa (tokens, embeddings, atención) y demostrar que dominas el arte del **prompt**. Todo se juega y se califica en un solo archivo: `dashboard-semana2.html`.

Un solo intento

Antes de empezar, el dashboard pide tu **nombre, equipo e integrantes**. La nota final se calcula **UNA sola vez** (al pulsar «Fase C»); después el dashboard se bloquea y debes **descargar el PDF** del reporte con aciertos y errores.

A Teoría de la semana (¡sale en el quiz!)

Cómo «piensa» una IA generativa

- **Token:** la unidad mínima de texto. «casa» $\approx$ 1 token; «increíblemente» puede ser 3. Los modelos cuentan y predicen por tokens, no por palabras.
- **Embedding:** convertir palabras en listas de números para que la máquina capte relaciones: `rey - hombre + mujer  $\approx$  reina`.
- **Atención:** el mecanismo que decide **qué palabras importan más** para predecir la siguiente.

Ingeniería de prompts (el arte de pedir)

- Escribe con **rol + contexto + restricciones + formato**. Un prompt vago = respuesta vaga.
- **Alucinación:** la IA inventa datos con seguridad total. No miente: completa texto «probable». ¡Verifica siempre!
- **Sesgo:** el modelo hereda prejuicios de datos mal balanceados.
- **Ética:** si el prompt es discriminatorio, el resultado también. Somos responsables del uso.

Lee `tools/05-vocabulario-ia.md` (filas de la Semana 2). El quiz del dashboard sale **exactamente** de aquí.

B Cómo funciona el dashboard

Abre `dashboard-semana2.html` (doble clic o arrástralo al navegador). Tiene 3 fases y las tres cuentan para la nota:

Fase	Qué es	Peso
 Fase A — Evaluación Teórica	3 preguntas de opción múltiple (Tokens, Alucinaciones, Sesgos). El dashboard califica solo: necesitas $\geq$ 2 aciertos para desbloquear la Fase B.	35 % de la nota
 Fase B — Generador de Prompts	El botón «  Asignar escenario» sortea un reto ético/práctico. Los 4 integrantes redactan UN prompt perfecto. El panel derecho da feedback en vivo con el checklist de ingeniería de prompts.	65 % de la nota

Fase C — Calificación final	Combina teoría + estructura del prompt → nota sobre 100. Al calcularla se bloquea el intento y se genera tu reporte en PDF (equipo, escenario, prompt y aciertos/errores).	—
------------------------------------	---	---

⚠ Cuidado

Si cierras el dashboard antes de calcular la nota, puedes seguir. Pero una vez calculada la nota final, **ya no hay vuelta atrás**.

C Reparto de roles para el prompt (equipo de 4)

Para que nadie se quede sin aportar su parte del prompt:

Integrante	Se encarga de...	Es la parte...
A	Definir quién es la IA («actúa como...»)	Rol
B	Explicar situación, tema y datos clave	Contexto
C	Qué NO debe hacer, límites, tono	Restricciones
D	Cómo quieres la salida y revisar el prompt completo	Formato + cierre

D Ejercicio · Escribe el prompt aquí (borrador en equipo)

Escenario sorteado: _____

Nuestro prompt:

Checklist antes de pulsar «evaluar»: ¿tiene **rol**? ¿**contexto**? ¿**restricciones**? ¿**formato de salida**? ¿permite un **tono y audiencia** claros?

E Entregables

- semana-02/ENTREGA.md con equipo e integrantes (rol de cada quien), la **nota final y desglose** (teoría vs. prompt), el **prompt** copiado tal cual y una **reflexión de 3 líneas**.
- El **reporte PDF** (reporte-semana2.pdf) dentro de semana-02/.
- Captura de la Fase A aprobada (evidencia de la teoría).

F Rúbrica (20 puntos)

Criterio	Puntos
Fase A: respuestas correctas de teoría	5

Fase B: el prompt incluye rol + contexto + restricciones	8
Fase B: calidad/creatividad y elementos avanzados	4
Reporte PDF + ENTREGA.md completos	3

G Autochequeo del equipo

- ☐ Entiendo qué es un **token** con un ejemplo propio.
- ☐ Puedo explicar por qué la IA **alucina**.
- ☐ Sé qué es un **sesgo** y de dónde viene.
- ☐ Mi prompt tiene **rol, contexto y restricciones** (y lo comprobó el panel).
- ☐ Descargué el **reporte PDF** y está en la carpeta.
- ☐ ENTREGA.md subido en la rama `equipo-N-semana-02`.



Reto extra (opcional)

Cambien UNA palabra del prompt (por ejemplo, el rol) y comparen la respuesta de la IA. ¿Cuánto cambió? Anótenlo en la reflexión.



Regresión Lineal: adivina tu calificación

Misión: enseñarle a una computadora a **predecir un número** (tu nota) a partir de tus horas de estudio. Tu primer modelo de Machine Learning real.

🎯 ¿Qué es la regresión lineal? (en 3 líneas)

Pones los datos en una gráfica y trazas la **recta que mejor pasa por en medio**. Esa recta te deja predecir valores que nunca viste. La fórmula es:

```
# la máquina encuentra estos dos números solita
calificacion = 2.5 + 6.0 × horas
```

Ese 6.0 (la **pendiente**) significa «cada hora extra sube 6 puntos». Ese es todo el secreto. ✨

📁 Archivos de esta semana

Archivo	Para qué
reto_regresion_lineal.py	El reto como script (córrelo con python)
reto_regresion_lineal.ipynb	El mismo reto en notebook (Google Colab / Jupyter)

⚠️ Los dos tienen el mismo contenido. Elige el que prefieras (tools/03-notebooks-y-scripts.md ayuda a decidir).

🚩 Cómo resolver el reto

Abre el archivo y busca las marcas 🚩 TODO. Son 10 pasos; el 4 en adelante los haces tú:

Paso	Qué hacer	Déjalo en TODO?
1-3	Importar herramientas, crear el dataset y graficarlo	Ya está
4	Separar X e y	← tú
5	train_test_split	← tú
6	Crear y entrenar el modelo	← tú
7	Predecir y medir MAE / R²	← tú
8	Interpretar coeficiente e intercepto	← tú
9	Dibujar la recta y guardar figura_semana3.png	← tú
10	🔥 Reto extra: añadir horas_sueno como segunda feature	Opcional

💡 Dato útil

Cuando un `todo(...)` te detiene con « TODO pendiente», NO es un error: es la señal de dónde debes escribir tu código.

Resultados del equipo (anota aquí tus números)

MAE (error promedio)	_____ puntos	R²	_____
Pendiente	_____	Intercepto	_____

Reflexión: la pendiente significa ... · el intercepto significa ...

Gráfica guardada en: `figura_semana3.png` ✓/X

Autochequeo (contesta en `ENTREGA.md`)

1. El intercepto significa...
2. Separamos `train/test` porque...
3. Con `test_size=0.9` pasaría... y con `0.05`...
4. Nuestro MAE fue __ y R^2 __
5. ¿Serviría el modelo para predecir precios de casas? ...

Rúbrica (20 puntos)

Criterio	Puntos
El código corre sin errores y completa los TODO	8
Mide y reporta MAE y R^2 correctamente	5
Interpreta pendiente/intercepto con sentido	4
Gráfica guardada + <code>ENTREGA.md</code> completo	3

✅ Autochequeo rápido

- ☐ Entiendo qué es una feature (X) y una respuesta (y).
- ☐ Sé por qué separo datos de entrenamiento y de prueba.
- ☐ Puedo explicar qué significa mi pendiente.
- ☐ Tengo mi gráfica guardada como evidencia.
- ☐ `ENTREGA.md` subido en la rama `equipo-N-semana-03`.

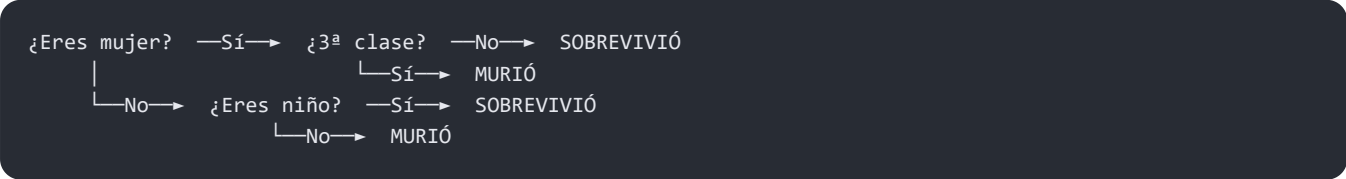
 Reto relámpago

Antes de codificar, dibujen a mano una nube de puntos y una recta. Luego comparen con la que saca la máquina.

Misión: entrenar un modelo que decida «¿sobreviviría esta persona?» usando un árbol de decisiones, e **interpretar** cómo razona.

🎯 ¿Qué es un árbol de decisión?

Es el juego de las **20 preguntas** convertido en IA. El modelo aprende cuál pregunta separa mejor a los sobrevivientes de los que no:



Ventaja enorme: puedes **ver** y **explicar** por qué decidió lo que decidió.

🚩 Los pasos (busca los TODO en el código)

Paso	Qué hacer	Déjalo en TODO?
1-2	Cargar el dataset (<code>sns.load_dataset("titanic")</code>) y explorar supervivencia por sexo y clase	Ya está
3	Preparar datos: elegir features, convertir <code>sex</code> a números, manejar los <code>age</code> nulos	← tú
4	Separar X / y y <code>train_test_split</code>	← tú
5	Crear el árbol con <code>max_depth=3</code> y entrenarlo	← tú
6	Evaluar: <code>accuracy_score</code> , <code>confusion_matrix</code> , <code>classification_report</code>	← tú
7	Dibujar el árbol con <code>plot_tree</code> y guardarlo (<code>arbol_titanic.png</code>)	← tú
8	🔥 Reto: comparar <code>max_depth</code> de 1 a 10 y ver el sobreajuste	Opcional

💡 **Sobreajuste**

Si el árbol es muy profundo «memoriza» los datos de entrenamiento y falla con datos nuevos. Verás exactamente eso en la tabla del reto.

Si el árbol es muy profundo «memoriza» los datos de entrenamiento y falla con datos nuevos. Verás exactamente eso en la tabla del reto.

Resultados del equipo

Accuracy del árbol	_____ %	Baseline («todos mueren»)	~62 %
Primera pregunta (raíz)	¿...?		
¿Por qué la eligió el árbol?			

Sobreajuste (reto `max_depth`)

Profundidad	Accuracy train	Accuracy test
1		
3		
10		
sin límite		

Conclusión sobre el sobreajuste que observamos: ...

Autochequeo del equipo

- ☐ Sé qué es una «hoja» y la «raíz» del árbol.
- ☐ Puedo explicar por qué convertimos `sex` a 0/1.
- ☐ Entiendo el sobreajuste y lo vi en la tabla.
- ☐ Mi árbol está guardado como imagen.
- ☐ ENTREGA.md subido en la rama `equipo-N-semana-04`.

Rúbrica (20 puntos)

Criterio	Puntos
Preprocesamiento correcto (nulos y texto→número)	5
El modelo entrena y se evalúa bien	6
Árbol dibujado e interpretado	6
ENTREGA.md + reflexión individual	3

Reto relámpago

Antes de correr el código, apuesten: ¿qué 3 columnas creen que predecirán mejor la supervivencia? Comparen con lo que eligió el árbol.



KNN: recomendador de películas

Misión: construir un sistema que, dada una película que te gusta, te recomiende otras parecidas. Tu primer «Netflix casero» 🎬.

La idea

KNN = K-Nearest Neighbors («los K vecinos más cercanos»). Para decidir algo sobre una película, miramos las **K más parecidas** y nos fijamos en ellas.

- Para **recomendar** → devolvemos las vecinas.
- Para **predecir el rating** → promediamos el rating de las vecinas.

⚠ Concepto clave: escalar los datos

Si `year` va de 1990 a 2020 y `accion` de 0 a 10, ¿el año domina la distancia! El modelo ignoraría los géneros. Con `StandardScaler` ponemos todo en la misma escala. **Este es el aprendizaje más importante de la semana.**

Los pasos (catálogo de 16 películas ya viene dentro del código)

Paso	Qué hacer	Déjalo en TODO?
1	Catálogo de películas	Ya está
2	Elegir features (<code>year</code> , <code>accion</code> , <code>comedia</code> , <code>romance</code>)	← tú
3	Escalar con <code>StandardScaler</code>	← tú
4	Encontrar vecinas con <code>NearestNeighbors</code>	← tú
5	Predecir rating con <code>KNeighborsRegressor</code>	← tú
6	Programar la función <code>recomendar(titulo, k)</code>	← tú
7	🔥 Reto: comparar <code>k=1</code> , <code>k=3</code> , <code>k=7</code>	Opcional

Resultados del equipo

Recomendaciones

Si te gustó **Matrix**, mira: _____ porque _____

Nuestra película favorita: _____ → recomendadas: _____

Resultados del modelo

MAE del regresor KNN	_____	¿Cambia con <code>k=7</code> ?	_____
----------------------	-------	--------------------------------	-------

Reflexión: el escalado cambió las recomendaciones porque ...

✓ Autochequeo del equipo

- ☐ Explico por qué el escalado es obligatorio en KNN.
- ☐ Entiendo la diferencia entre `fit_transform` y `transform`.
- ☐ Mi función `recomendar()` no se recomienda a sí misma.
- ☐ Probé varios valores de `k`.
- ☐ `ENTREGA.md` subido en la rama `equipo-N-semana-05`.

Rúbrica (20 puntos)

Criterio	Puntos
Escala correctamente (fit/transform bien usados)	5
Encuentra vecinos y recomienda con la misma escala	6
Entrena y evalúa el regresor KNN	5
<code>ENTREGA.md</code> con reflexión individual	4

Reto relámpago

¿Qué características añadirías al catálogo para mejorar las recomendaciones (actor favorito, saga, duración)? Anótenlas para la clase.



K-Means: segmentación de clientes

Misión: agrupar 240 clientes en «tipos de persona» **sin que nadie te diga cuáles son los grupos**. Descubre los patrones por tu cuenta. 🔍

🎯 ¿Qué es K-Means y qué lo hace distinto?

K-Means agrupa puntos parecidos alrededor de **centroides** (los promedios del grupo). A diferencia de las semanas 3–5, aquí **no hay respuestas correctas**: es **aprendizaje no supervisado**.

Supervisado (S3–S5)	No supervisado (S6)
Con etiquetas	Sin etiquetas
Predice	Descubre grupos

🚩 Los pasos

Paso	Qué hacer	Déjalo en TODO?
1–2	Generar clientes con <code>make_blobs</code> y graficarlos (son ficticios)	Ya está
3	Escalar	← tú
4	Método del codo para elegir k	← tú
5	Entrenar K-Means y pintar los grupos	← tú
6	Interpretar y nombrar cada segmento	← tú
7	🔥 Reto: <code>silhouette_score</code> y comparar con los grupos «secretos»	Opcional

Resultados del equipo

Elección de k

El codo se ve en **k=**__. Elegimos **k=**__ porque _____

Gráficas a guardar: `codo_kmeans.png` y `clusters_clientes.png` ✓/X

Nuestros segmentos (usa el `groupby(...).mean()`)

Cluster	Edad promedio	Gasto promedio	Visitas	Nombre que le pusimos
0				
1				
2				

Justifica cada nombre con los promedios reales: ...

✓ Autochequeo del equipo

- ☐ Sé qué es un centroide.
- ☐ Puedo explicar por qué necesitamos escalar (otra vez).
- ☐ Mi codo tiene una explicación, no solo «porque sí».
- ☐ Los nombres de mis clusters se basan en los promedios reales.
- ☐ ENTREGA.md subido en la rama `equipo-N-semana-06`.

Rúbrica (20 puntos)

Criterio	Puntos
Escalado correcto	4
Codo bien interpretado	5
Clusters graficados y coherentes	5
Nombres de segmentos justificados con los promedios	6

Reto relámpago

¿Para qué crees que una tienda usa esto? Escribe un ejemplo de campaña distinta para cada uno de tus segmentos.



Redes Neuronales: leer dígitos (MNIST)

Misión: construir y entrenar tu primera **red neuronal** y ponerla a leer números escritos a mano. El clásico «hola mundo» de la IA.

🎯 ¿Qué es una red neuronal (versión simple)?

Un equipo de **neuronas diminutas** organizadas en capas. Cada pixel entra, cada neurona multiplica por **pesos** y vota, y la última capa elige el dígito más probable. Entrenar = ajustar esos pesos para acertar.

pixeles → [capa oculta de neuronas] → "es un 7" (81 %)

Usamos `MLPClassifier` de scikit-learn para entender el concepto sin pelearse con frameworks pesados. En producción se usa TensorFlow/PyTorch, pero la idea (capas + pesos + votos) es la misma.

⚠ Internet

`fetch_openml("mnist_784")` descarga internet la primera vez. Si falla, el código tiene **plan B**: usa `load_digits` (8×8), que ya viene instalado. Ideal para Google Colab.

🚩 Los pasos

Paso	Qué hacer	Déjalo en TODO?
1-2	Cargar MNIST (con plan B) y ver los dígitos como imágenes	Ya está
3	Normalizar los pixeles a 0-1	← tú
4	Separar train/test con stratify	← tú
5	Definir la red (<code>MLPClassifier</code> , capas ocultas)	← tú
6	Entrenar	← tú
7	Evaluar: accuracy + matriz de confusión	← tú
8	Reto: probar arquitecturas (32,), (64,), (128, 64), (256, 128, 64)	Opcional

Resultados del equipo

Mejor arquitectura: _____ con `max_iter=`__ · Accuracy: _____ %

Dígitos que más confundimos: _____ con _____

Experimentos

Arquitectura	max_iter	Accuracy	Tiempo aprox.
(32,)	20		
(64,)	20		
(128, 64)	30		
(256, 128, 64)	30		

Conclusión sobre qué arquitectura funcionó mejor y por qué: ...

Entregables de imágenes: 1 de dígitos de ejemplo + 1 de errores ✓/✗

✓ Autochequeo del equipo

- ☐ Sé cuántos valores de entrada tiene cada imagen y por qué.
- ☐ Puedo explicar qué hace una neurona ($\text{peso} \times \text{entrada} + \text{voto}$).
- ☐ Entiendo para qué normalizamos.
- ☐ Comparé al menos 3 arquitecturas y saqué conclusiones.
- ☐ ENTREGA.md subido en la rama `equipo-N-semana-07`.

Rúbrica (20 puntos)

Criterio	Puntos
Normalización y split con <code>stratify</code> correctos	4
Red definida y entrenada sin errores	6
Evaluación con accuracy + matriz de confusión	5
Experimentación de arquitecturas con conclusión	5

Reto relámpago

¿Qué dígitos se confunden más entre sí (4/9, 3/8, 1/7)? Dibújalos lado a lado y verás que ¡a ti también te cuestan!

NLP: análisis de sentimiento

Misión: enseñarle a una computadora a distinguir una reseña **positiva** de una **negativa**. La base de los filtros de spam y de los moderadores de comentarios. 🗣️

🎯 ¿Qué es NLP?

Natural Language Processing = hacer que la máquina trabaje con texto humano. El problema central: las computadoras entienden **números**, no letras. Así que convertimos texto → números (**vectorizar**) y clasificamos.

Vectorizador	Qué hace
<code>CountVectorizer</code>	Cuenta cuántas veces aparece cada palabra
<code>TfidfVectorizer</code>	Cuenta, pero resta peso a palabras comunes («de», «la»)

⚠️ Regla de oro (¡sale en la rúbrica!)

El `fit` del vectorizador se hace SOLO con el texto de entrenamiento. Si no, sería «hacer trampa».

🚩 Los pasos

Paso	Qué hacer	Déjalo en TODO?
1	Dataset de reseñas	Ya está
2	Separar train/test	← tú
3	Vectorizar con <code>TfidfVectorizer</code>	← tú
4	Entrenar <code>MultinomialNB</code>	← tú
5	Evaluar (accuracy + matriz de confusión)	← tú
6	Probar reseñas nuevas escritas por el equipo	← tú
7	🔥 Reto: palabras más positivas y más negativas	Opcional

Resultados del equipo

Accuracy del modelo: _____ %

Palabras positivas: _____ · Palabras negativas: _____

Nuestras reseñas nuevas (2 por cada integrante)

Reseña	Predicción	¿Estuvimos de acuerdo?

Reflexión: el sarcasmo es difícil de detectar porque ...

Autochequeo del equipo

- ☐ Puedo explicar por qué el texto se convierte en números.
- ☐ Sé la diferencia entre `fit_transform` y `transform`.
- ☐ Añadí 2 reseñas propias al dataset.
- ☐ Encontré las palabras «deladoras» de cada sentimiento.
- ☐ `ENTREGA.md` subido en la rama `equipo-N-semana-08`.

Rúbrica (20 puntos)

Criterio	Puntos
<code>fit/transform</code> correctamente usados (sin trampa)	6
Modelo entrenado y evaluado	5
Predicción de reseñas nuevas funcionando	5
Aportación individual (2 reseñas por integrante) + reflexión	4

Reto relámpago

Escriban una reseña con **sarcasmo** («qué maravilla, se rompió a la primera»). ¿El modelo acierta? ¿Por qué el sarcasmo es difícilísimo para la IA? Guárdenlo para la discusión de la Semana 9.



Tu portafolio web con Next.js

Misión: convertirte en desarrollador/a web frontend. Vas a construir tu **portafolio personal** con Next.js + TypeScript para mostrar todo lo que hiciste en el curso. Al final tendrá su propio link en internet.



¿Qué es Next.js y por qué no HTML a secas?

En la Semana 1 usaste HTML directo: un archivo = una página. Está bien para empezar, pero en proyectos grandes necesitas **componentes** (piezas reutilizables) y TypeScript (que te avise de errores antes de ejecutar).

Concepto	En cristiano
Componente	Una función que devuelve HTML (una tarjeta, un botón, la página entera)
Props	Los datos que le pasas a un componente (<code><ProyectoCard proyecto={p} /></code>)
TypeScript	JavaScript con tipos: si te equivocas, te lo dice el editor
JSX / TSX	Escribir HTML dentro de TypeScript



Dónde está el proyecto

El proyecto vive en la carpeta `web/` (compartida con la Semana 10). Si te atascas: `tools/04-nextjs-y-vercel.md`.

```
cd web
npm install    # solo la primera vez
npm run dev    # abre http://localhost:3000
```



Los pasos de esta semana

Abre los archivos y busca los TODO:

Archivo	Qué cambiar
<code>data/proyectos.ts</code>	Cambia los 8 ejemplos por tus proyectos reales (los de las semanas 1–8). Ponles tu emoji, título y descripción.
<code>app/page.tsx</code>	Pon tu nombre , tu frase y tus links .
<code>app/layout.tsx</code>	Cambia el <code>title</code> (lo que sale en la pestaña).
<code>app/globals.css</code>	Personaliza colores y fuentes. Hazlo TUYO.
<code>components/ProyectoCard.tsx</code>	Mejora el diseño de las tarjetas.
—	Corre <code>npm run build</code> y comprueba que no haya errores .

Resultados del equipo

Integrante	Qué cambió en el portafolio

Salida final de `npm run build`: `✓ Compiled successfully ✓/X`

Captura del portafolio: `captura-portafolio.png ✓/X`

Autochequeo del equipo

- ☐ Puedo explicar qué es un componente y para qué sirven las props.
- ☐ Cambié el título en `layout.tsx` y se ve en la pestaña.
- ☐ Mi `data/proyectos.ts` tiene MIS proyectos, no los de ejemplo.
- ☐ `npm run build` termina sin errores.
- ☐ `ENTREGA.md` subido en la rama `equipo-N-semana-09`.

Rúbrica (20 puntos)

Criterio	Puntos
Portafolio personalizado (nombre, frase, links reales)	6
Proyectos propios cargados desde <code>data/proyectos.ts</code>	5
Edición de componentes y estilos	5
<code>npm run build</code> sin errores + <code>ENTREGA.md</code>	4

Reto relámpago

Añade una sección nueva « Mis hobbies» usando un componente propio. ¿Te animas a que sea reutilizable con props?

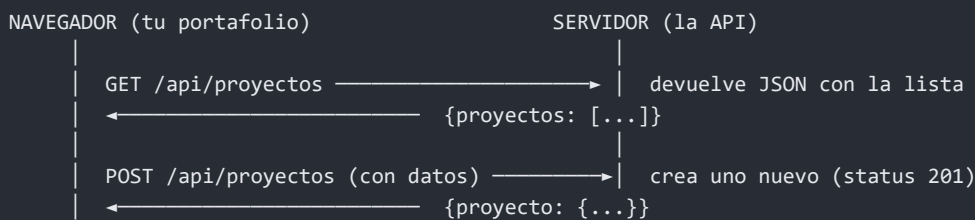


Conecta una API y publica tu portafolio en Vercel

Misión final: que tu portafolio **pida y envíe datos a una API** (como lo hacen las apps de verdad) y luego **publicarlo en internet** con un link público para compartirlo con el mundo. 🌐

🎯 ¿Qué vas a aprender?

Cómo dos programas conversan por internet con **REST**:



Método	Para qué
GET	Leer / traer datos
POST	Crear algo nuevo
DELETE	Borrar
PUT / PATCH	Actualizar

🛠️ Lo que ya está montado (míralo antes de tocar nada)

Archivo	Qué enseña
web/app/api/proyectos/route.ts	El «backend» simulado (GET y POST)
web/lib/api.ts	Las funciones que llaman a la API (fetch)
web/components/ProyectosSection.tsx	Estados de carga, error y recarga
web/components/ProyectoForm.tsx	El formulario que hace POST

🔍 Truco de detective

Abre la pestaña **Network** (o Red) del navegador, agrega un proyecto desde el formulario y **mira la petición POST**. Verás el JSON viajando. ¡Eso es una API en vivo!

🚩 Los pasos

1. `npm run dev` y agrega un proyecto con el **formulario** (mira la petición en Network).
2. Lee `app/api/proyectos/route.ts` y contesta: ¿qué pasa si falta el título?

3.  **Reto de código:** implementa el método **DELETE** creando `app/api/proyectos/[id]/route.ts` y un botón «» en la tarjeta.
4.  **Reto extra:** un filtro por tecnología o un buscador.
5. **Despliega en Vercel** (`tools/04-nextjs-y-vercel.md`) y comparte el link.

Lo más importante de la semana

El **link público de Vercel** funcionando. Compártelo en la clase, no en localhost.

Resultados del equipo

 **Link de producción:** `https://tu-portafolio.vercel.app`

Evidencia: captura del `GET /api/proyectos` ✓/X · captura del `POST` desde el formulario ✓/X

Reto extra implementado: _____ (DELETE / filtro / buscador)

¿Por qué lo implementamos así? ...

✓ Autochequeo del equipo

- ☐ Entiendo la diferencia entre `GET` y `POST`.
- ☐ Sé qué es un status `200`, `201` y `400`.
- ☐ Mi app muestra «Cargando...» y maneja el error si la API falla.
- ☐ Mi portafolio está **público en Vercel**.
- ☐ `ENTREGA.md` subido en la rama `equipo-N-semana-10`.

Rúbrica (20 puntos)

Criterio	Puntos
El portafolio carga proyectos desde la API (GET)	5
El formulario crea proyectos (POST) y muestra estados	5
Desplegado en Vercel y link público funcionando	6
Reto extra (DELETE / filtro) + <code>ENTREGA.md</code> completo	4

Cierre del curso

Comparte tu link con la clase y explica en 3 minutos cuál fue **tu** aportación a cada fase (desmitificación, ML, web). ¡Lo lograste!